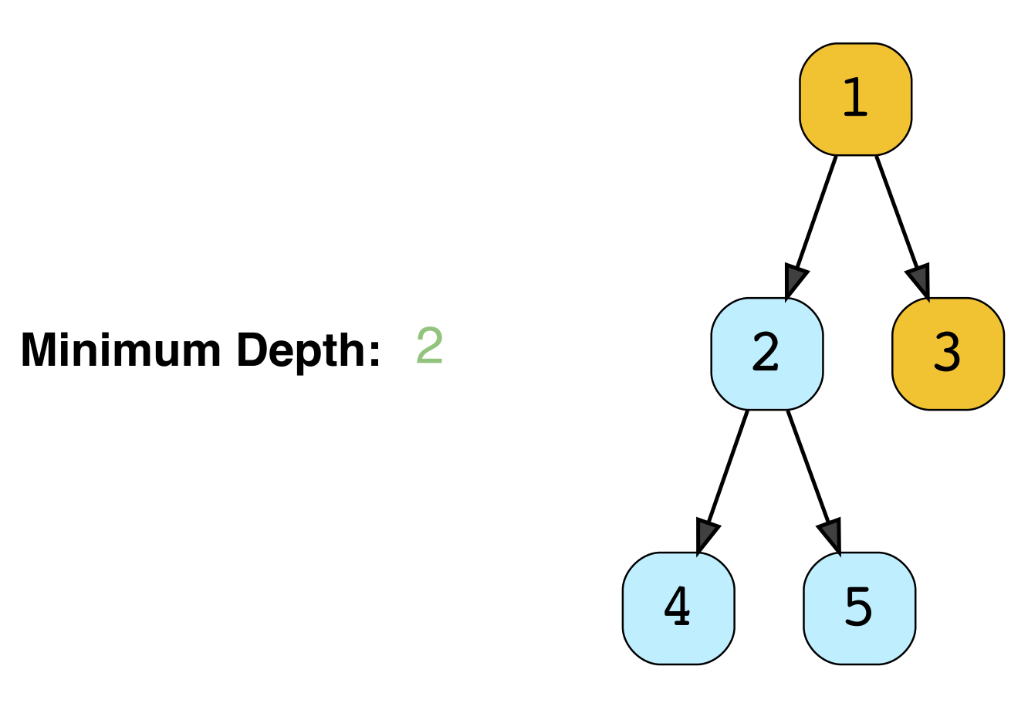
Solution: Minimum Depth of a Binary Tree

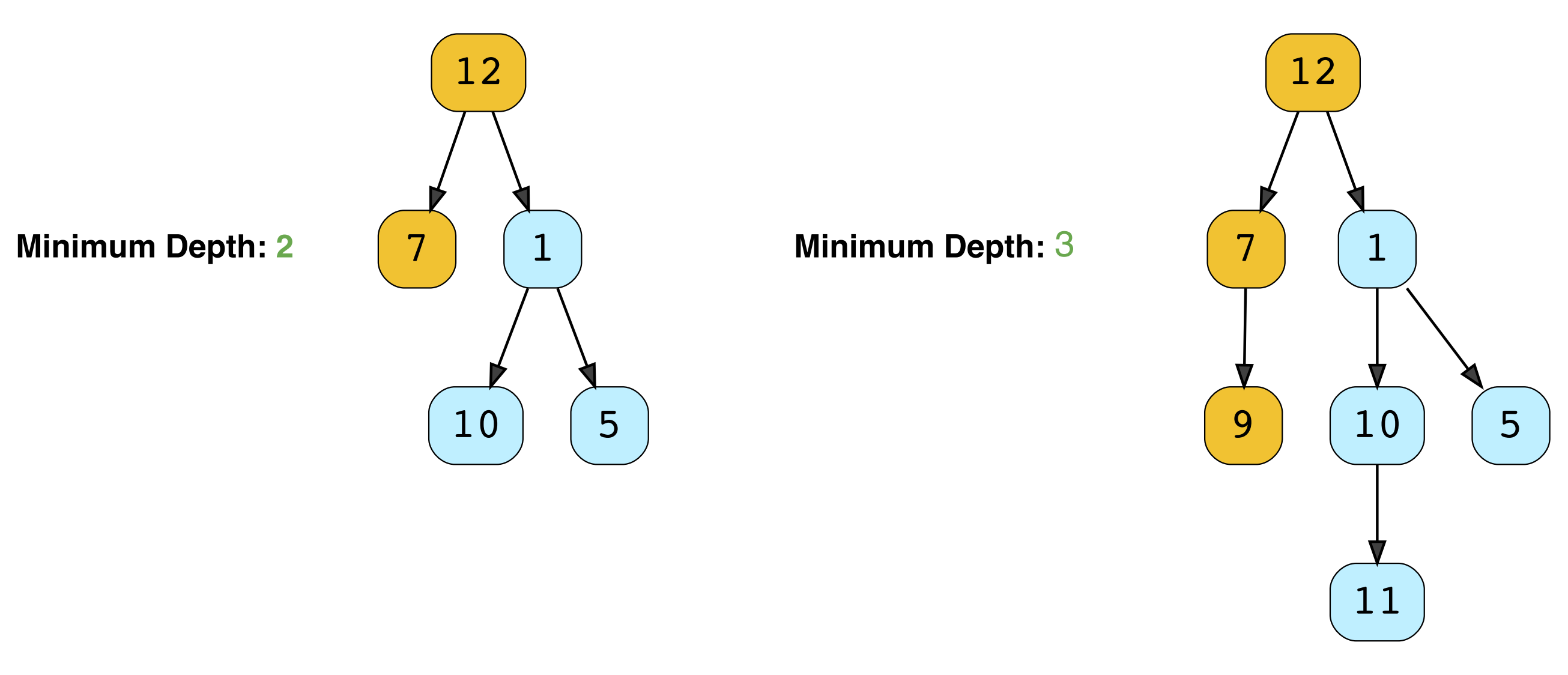
Problem Statement

Find the minimum depth of a binary tree. The minimum depth is the number of nodes along the **shortest path from the root node to the nearest leaf node.**

Example 1:


Minimum Depth: 2
1
2
3
4
5

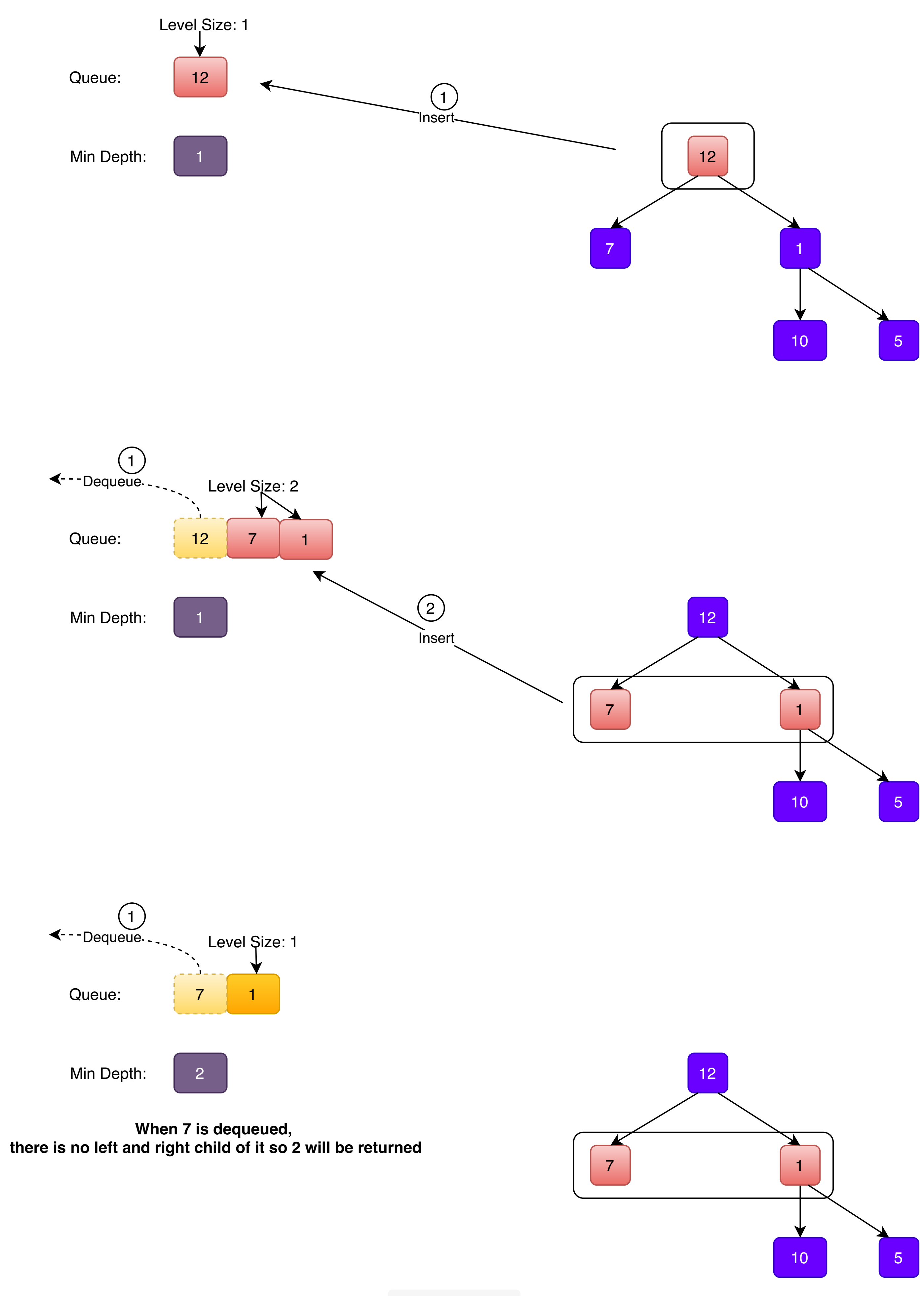
Example 2:


Minimum Depth: 2
12
7
1
10
5
Minimum Depth: 3
12
7
1
9
10
5
11

Solution

This problem follows the **Binary Tree Level Order Traversal** pattern. We can follow the same **BFS** approach. The only difference will be, instead of keeping track of all the nodes in a level, we will only track the depth of the tree. As soon as we find our first leaf node, that level will represent the minimum depth of the tree.

Here is the visual representation of the algorithm:

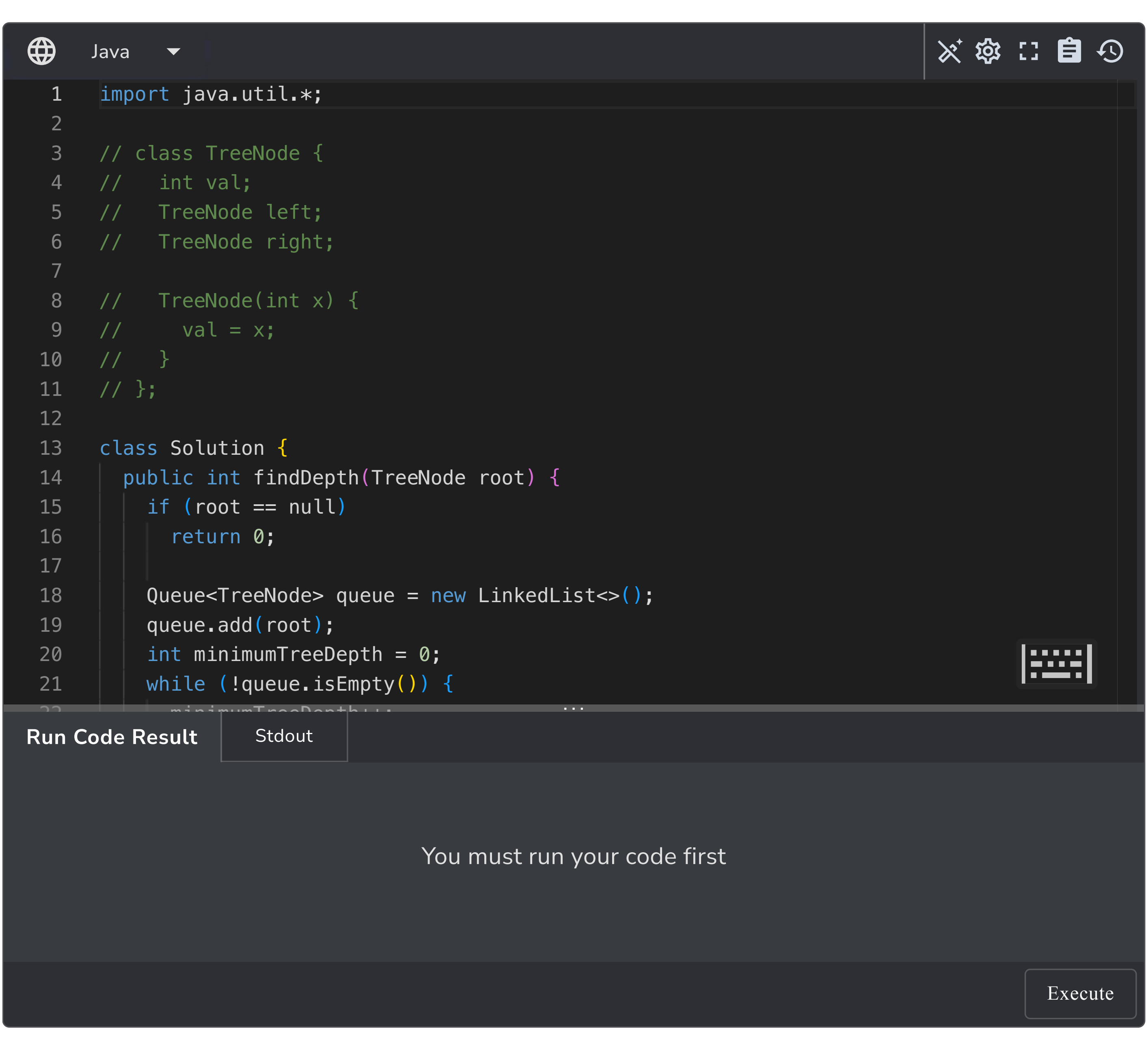

Level Size: 1
Queue:
12
Min Depth:
1
① Insert
12
7
1
10
5
① Dequeue
Level Size: 2
Queue:
12
7
1
Min Depth:
1
② Insert
12
7
1
10
5
① Dequeue
Level Size: 1
Queue:
7
1
Min Depth:
2
12
7
1
10
5

**When 7 is dequeued,
there is no left and right child of it so 2 will be returned**

Min Depth BT

Code

Here is the code for this algorithm:


Java
Run Code Result
Stdout
Execute

```java
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public int findDepth(TreeNode root) {
    if (root == null)
      return 0;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.add(root);
    int minimumTreeDepth = 0;
    while (!queue.isEmpty()) {
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is O(N), where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

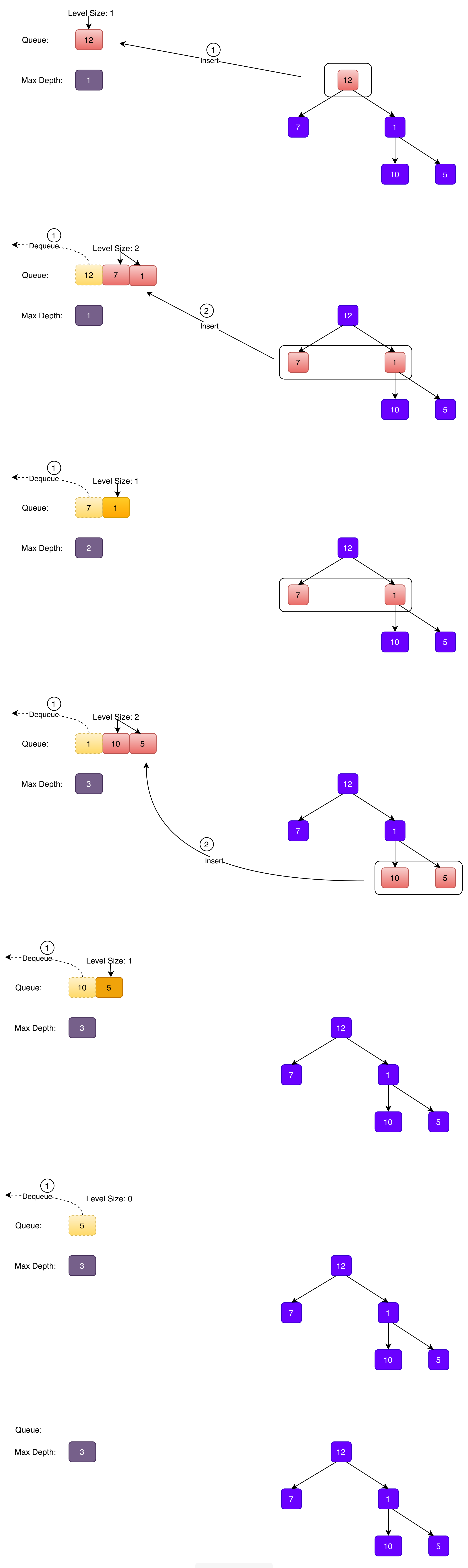
The space complexity of the above algorithm will be O(N) which is required for the queue. Since we can have a maximum of N/2 nodes at any level (this could happen only at the lowest level), therefore we will need O(N) space to store them in the queue.

Similar Problems

Problem 1: Given a binary tree, find its maximum depth (or height).

Solution: We will follow a similar approach. Instead of returning as soon as we find a leaf node, we will keep traversing for all the levels, incrementing `maximumDepth` each time we complete a level. Here is what the code will look like:

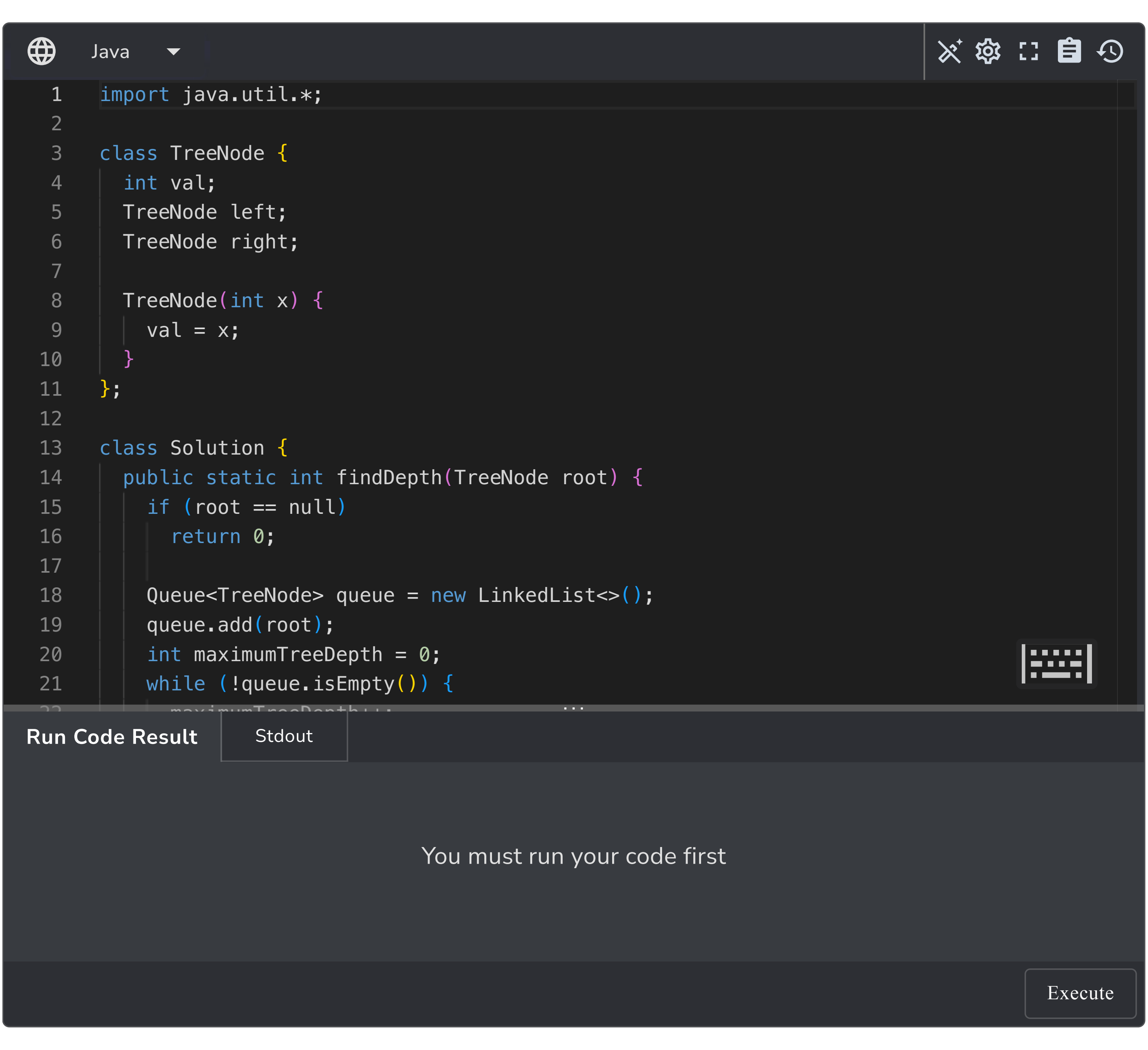
Here is the visual representation of the algorithm:


Level Size: 1
Queue:
12
Max Depth:
1
① Insert
12
7
1
10
5
① Dequeue
Level Size: 2
Queue:
12
7
1
Max Depth:
1
② Insert
12
7
1
10
5
① Dequeue
Level Size: 1
Queue:
7
1
Max Depth:
2
12
7
1
10
5
① Dequeue
Level Size: 2
Queue:
1
10
5
Max Depth:
3
② Insert
12
7
1
10
5
① Dequeue
Level Size: 1
Queue:
10
5
Max Depth:
3
12
7
1
10
5
① Dequeue
Level Size: 0
Queue:
5
Max Depth:
3
12
7
1
10
5
Queue:
Max Depth:
3
12
7
1
10
5

Max. Depth BT

Code

Here is the code for this algorithm:


Java
Run Code Result
Stdout
Execute

```java
import java.util.*;

class TreeNode {
  int val;
  TreeNode left;
  TreeNode right;

  TreeNode(int x) {
    val = x;
  }
};

class Solution {
  public static int findDepth(TreeNode root) {
    if (root == null)
      return 0;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.add(root);
    int maximumTreeDepth = 0;
    while (!queue.isEmpty()) {
```

You must run your code first

← Previous
Minimum Depth Of A Binary Tree (Easy)

Next →
Level Order Successor (Easy)

☐ Mark as Completed